Some important rules related to checked Exception :

a) If the try block does not throw any checked exception then in the corresponding catch block we can't handle checked exception it will generate compilation error i.e "exception never thrown from the corresponding try statement"

Example :-

```
public class Test
{
    public static void main(String[] args)
    {
        try
        {
          //Try Block is not throwing checked exception
          //i.e. InterruptedException
        }
        catch (InterruptedException e).//error
        {
        }
    }
}
```

Note :- The above rule is not applicable for Unchecked Exception

```
        try
        {
        }
        catch(ArithmeticException e)  //Valid
        {
            e.printStackTrace();
        }
```

b) If the try block does not throw any exception then in the corresponding catch block we can write Exception OR Throwable because both are the super classes for all types of Exception whether it is checked or unchecked.

```
package com.ravi.checked_exception_rules;

public class CatchingAllSuperClass
{
    public static void main(String[] args)
    {
        try
        {

        }
        catch(Throwable     e)  //Exception OR Throwable both are allowed
        {
          e.printStackTrace();
        }

    }
}
```

c) At the time of method overriding if the super class method does not reporting or throwing checked exception then the overridden method of sub class not allowed to throw checked exception otherwise it will generate compilation error but overridden method can throw Unchecked Exception.

```
package com.ravi.checked_exception_rules;

import java.io.IOException;
import java.io.FileNotFoundException;
import java.io.IOException;

class Super
{
    public void show()
    {
        System.out.println("Super class method not throwing checked Exception");
    }
}

class Sub extends Super
{
    @Override
    public void show() //throws ClassNotFoundException error
    {
        System.out.println("Sub class method should not throw checked Exception");
    }
}

public class MethodOverridingAllUnchecked {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

    }

}
```

d) If the super class method declare with throws keyword to throw a checked exception, then at the time of method overriding, the overridden method of sub class can't add more exceptions in the throws keyword.
Also Overridden method is also using throws keyword.
keyword to throw checked exception then it must be either SAME exception class or SUB class, it should not be SUPER class as well as we can't add more exceptions in the overridden method.

```
package com.ravi.checked_exception_rules;

import java.io.FileNotFoundException;
import java.io.IOException;

class Base
{
    public void show() throws FileNotFoundException
    {
        System.out.println("Super class method ");
    }
}
class Derived extends Base
{
    @Override
    public void show() //throws IOException error
    {
        System.out.println("Sub class method ");
    }
}

public class MethodOverridingAllThrows
{
    public static void main(String[] args)
    {
        System.out.println("Overridden method may or may not throw checked exception but if it is throwing then must be same or sub class");
    }

}
```

e) Just like return keyword, we can't use throw keyword inside static and non static block to throw an exception object because all initializers must be executed normally.

We can use throw keyword in the protection of try-catch as the code will be executed normally.

```
class ExceptionDemo
{
  static
  {
      try
      {
        throw new ArithmeticException();
      }
      catch (ArithmeticException e)
      {
        IO.println("Divide by zero problem");
      }
  }
  public static void main(String[] args)
  {
  }
}
```

f) Whenever we are **calling any method** and if the callee method throws any type of **checked exception** then due to exception propagation concept, exception object will be propagated to the caller method so caller method must need protection (either try-catch OR throws).

Case 1:

```
class ExceptionDemo
{
  public static void m1() throws ClassNotFoundException
  {
  }

  public static void main(String[] args)
  {
      m1();  //compulsory try-catch OR throws [throws not recommended]
  }
}
```

Case 2 :

```
class ExceptionDemo
{
    public static void m1() throws Exception    [If we write Exception OR Throwable then caller must need
                                                 protection]
    {
    }

    public static void main(String[] args)
    {
        m1();
    }
}
```

try with resources : (Automatic Closing Facility)

JDK 1.5
*** All my resources class must be close() method.**
interface java.io.Closeable
{
void close() throws IOException;
}
class XYZ implements Closeable
{
public void close()
{
}
}
XYZ x = new XYZ();
x.close(); //finally block

JDK 1.7
interface java.lang.AutoCloseable
{
void close() throws Exception;
}
How AutoCloseable will work :
*** In order to work with AutoCloseable we need to follow 2 steps :**
1) A resource class who wants AutoClosing facility must implements AutoCloseable interface.
2) We should pass the object of resource class as a parameter to the try block

To avoid all the limitation of finally block, Java software people introduced a separate concept i.e try with resources from java 7 onwards.

```
Case 1 :

try(resource1 ; resource2)  //Only the resources will be handled/closed
{
}

Case 2 :

//Resources and Exception both will be handled
try(resource1 ; resource2)
{
}
catch(Exception e)
{
}

Case 3 :

try with resources enhancement from java 9v

DatabaseResource d1 = new DatabaseResource();
FileResource f2 = new FileResource();

try(d1; f2)
{
}
catch(Exception e)
{
}
```

Note : The DatabaseResource class and FileResource class must implements either from Closeable or AutoCloseable.

There is a predefined interface available in java.lang package called AutoCloseable from JDK 1.7 which contains predefined abstract method i.e close() which throws Exception.

There is another predefined interface available in java.io package called Closeable from JDK 1.5, this Closeable interface is the sub interface for AutoCloseable interface.

```
public interface java.lang.AutoCloseable
{
  public abstract void close() throws Exception;
}

public interface java.io.Closeable extends java.lang.AutoCloseable
{
  void close() throws IOException;
}
```

Whenever we pass any resource class object, as part of try with resources as a parameter then that class must implements either Closeable or AutoCloseable interface so, try with resources will automatically call the respective class
close() method even an exception is encountered in the try block.

```
ResourceClass rc = new ResourceClass();
try(rc)
{
}
catch(Exception e)
{

}
```

This ResourceClass must implements either Closeable or AutoCloseable interface so, try block will automatically call the close() method as well as try block will get the guarantee of close() method support in the respective class.

The following program explains how try block is invoking the close() method available in DatabaseResource class and FileResource class.

```
package com.ravi.try_with_resources;

public class DatabaseResource implements AutoCloseable
{
    @Override
    public void close() throws Exception
    {
        IO.println("Database resource closed!!!");

    }

}


package com.ravi.try_with_resources;

import java.io.Closeable;
import java.io.IOException;

public class FileResource implements Closeable
{
    @Override
    public void close() throws IOException
    {
        IO.println("File resource closed !!!");
    }

}


package com.ravi.try_with_resources;

public class TryWithResources
{
    public static void main(String[] args) throws Exception
    {
        DatabaseResource d = new DatabaseResource();
        FileResource f = new FileResource();

        try(d; f; //closing right to left
        {
            IO.println(5/0);
        }
        catch(ArithmeticException e)
        {
            IO.println("Divide By Zero problem");
        }

    }

}
```

Another program on try with resources to close the Scanner class :

```
package com.ravi.try_with_resources;

import java.util.InputMismatchException;
import java.util.Scanner;

public class TryWithResourcesWithScanner {

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        try(sc) //calling the close method of scanner class
        {
            IO.println("Enter your Age :");
            int age = sc.nextInt();
            IO.println("Your age is :"+age);
        }
        catch(InputMismatchException e)
        {
            System.err.println("Input is invalid");
        }

    }

}
```

Note : While working with try with resources, automatically java compiler will insert finally block to close the resources.